DOCUMENTAÇÃO
DO
PROJETO PROCESSAMENTO DE PAGAMENTO

Objetivo do Projeto : Gateway de Pagamentos Simplificado

O projeto tem como objetivo simular um fluxo real de processamento de pagamentos utilizado em ambientes financeiros, onde usuários externos enviam requisições de pagamento e o sistema interno é responsável por validar saldo, processar a transação e retornar o status final da operação.

A proposta busca representar um cenário comum em grandes empresas e bancos, onde sistemas internos precisam ser disponibilizados de forma segura, escalável e resiliente para usuários externos.

Além disso, o projeto permite a simulação de eventos assíncronos com Kafka, aproximando a solução de uma arquitetura real utilizada no mercado financeiro.

1.Planejamento:

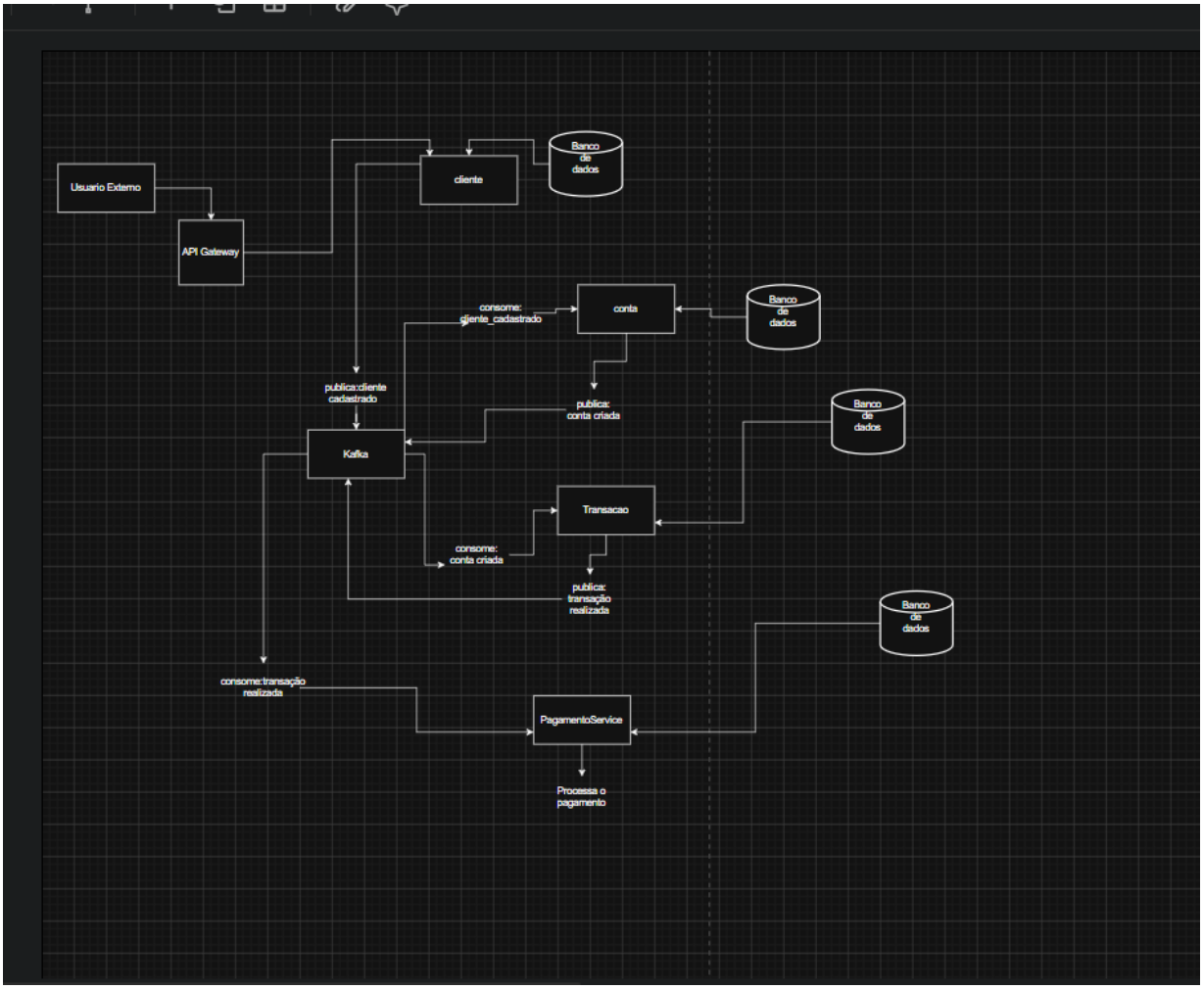
Este projeto resolve o problema de processamento de pagamentos em massa em sistemas financeiros.

Quando um grande volume de transações é enviado ao mesmo tempo, sistemas tradicionais tendem a ficar lentos, perder dados ou falhar.

A solução utiliza microsserviços para separar as responsabilidades, filas com Kafka para absorver o volume sem sobrecarregar o sistema e uma API Gateway como ponto de entrada seguro para os usuários externos.

O resultado é um sistema escalável, resiliente e preparado para evoluir conforme a demanda cresce.

2. ESTRUTURA/ARQUITETURA:



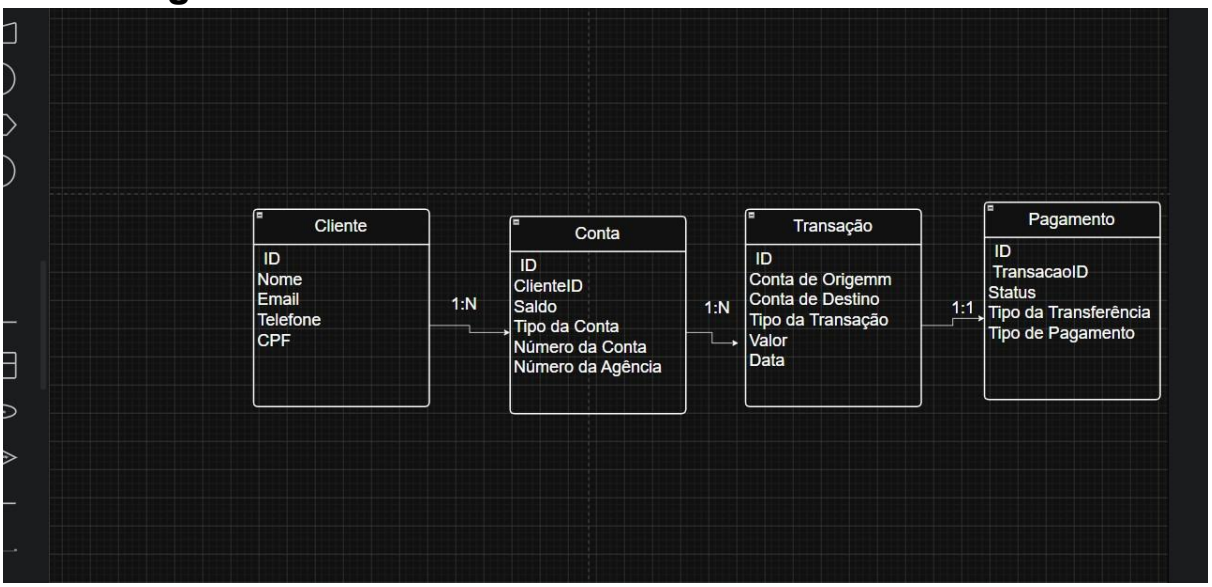
Explicação

O acesso de usuários externos ao sistema interno acontece inicialmente por meio da API Gateway, responsável por centralizar a entrada das requisições e atuar como ponto único de acesso entre o cliente e os microsserviços da aplicação.A API Gateway realiza o roteamento das requisições, controle de acesso, autenticação inicial, validação de segurança e direcionamento para os serviços corretos, evitando o acesso direto aos microsserviços internos.Ela se comunica com os serviços de Cliente, Conta, Transação e Pagamento, garantindo maior organização, segurança e desacoplamento da arquitetura.A comunicação entre os microsserviços ocorre principalmente de forma assíncrona com Kafka, utilizado para lidar com alto volume de eventos, retenção de mensagens e maior resiliência em cenários financeiros.Cada microsserviço possui seu próprio banco de dados, garantindo independência entre os serviços, redução de acoplamento e maior facilidade de escalabilidade e manutenção.

Fluxo

O fluxo principal do sistema funciona da seguinte forma:O ClienteService é responsável pelo cadastro e gerenciamento das informações do cliente, como nome, CPF, email e dados de identificação.Após o cadastro do cliente, essas informações são utilizadas pelo ContaService, responsável pela criação automática da conta bancária vinculada ao usuário.Após a criação da conta, o ContaService envia para o TransacaoService informações como:
accountI,dclientId,typeAccountO TransacaoService é responsável pelo gerenciamento das transações financeiras, recebendo dados como:**accountId,accountDestinyId,typeTransaction,amount**Com base nessas informações, a transação é criada e validada.Após isso, o PaymentService recebe:**transactionId,typeTransaction**Por fim, o PaymentService realiza o processamento do pagamento, confirmação da operação e atualização do status final da transação.Durante esse processo, eventos importantes são publicados no Kafka, permitindo comunicação assíncrona entre os serviços, auditoria de eventos e maior resiliência do sistema.

3. Modelagem:



Cliente → Conta (1:N)

Um cliente pode possuir uma ou várias contas bancárias, seja dentro da mesma instituição ou em diferentes contextos financeiros.

Por esse motivo, o relacionamento entre Cliente e Conta foi definido como 1:N, onde um único cliente pode estar vinculado a múltiplas contas.

Conta → Transação (1:N)

Uma conta bancária pode realizar diversas transações ao longo do tempo, como transferências, pagamentos, recebimentos e movimentações financeiras.Dessa forma, o relacionamento entre Conta e Transação foi definido como 1:N, onde uma única conta pode estar associada a várias transações.

Transação → Pagamento (1:1)

Cada transação gerada no sistema está associada a um único processo de pagamento, como PIX, TED, DOC ou boleto.

Por esse motivo, o relacionamento foi definido como 1:1, onde cada transação possui apenas um pagamento correspondente.

4. Segurança:

Explicação:

A etapa de segurança é responsável por garantir a proteção, a confiabilidade e a integridade do sistema financeiro.Será implementada a autenticação para validar a identidade do usuário e a autorização para definir quais ações cada perfil poderá executar dentro da plataforma.O uso de JWT permitirá a geração de tokens seguros para controle de sessão e acesso aos serviços protegidos da aplicação.Também serão aplicados mecanismos de auditoria para registrar operações importantes, antifraude para análise de transações suspeitas, idempotência para evitar pagamentos duplicados e estratégias de proteção contra falhas, acessos indevidos e sobrecarga do sistema, como rate limiting, retry e circuit breaker.

SecurityFilter:

O SecurityFilter intercepta cada requisição antes que ela chegue ao controller, extrai e valida o JWT enviado no header Authorization e registra o usuário autenticado no contexto do Spring Security, permitindo o controle de acesso às rotas protegidas.

TokenConfig:

Responsável por gerar e validar o token JWT utilizado na autenticação da aplicação.

O token é criado com base em informações seguras, como ID, email e role (perfil de acesso do usuário), além de possuir um tempo de expiração definido de 24 horas.Também realiza a validação do token em cada requisição protegida**.**

AuthConfig:

Responsável por localizar o usuário através do email e fornecer essas informações ao Spring Security para que o processo de autenticação seja realizado corretamente.

BCrypt:

Responsável pela criptografia segura das senhas antes de serem armazenadas no banco de dados, garantindo maior proteção contra vazamento de credenciais.

SecurityConfig:

Define as regras de segurança da aplicação, especificando quais rotas serão públicas, como cadastro e login, e quais exigirão autenticação e autorização, como rotas administrativas com permissões restritas.

JWTClienteData:

Record responsável por representar os dados do usuário extraídos do token JWT, como ID, email e permissões de acesso

Role

Enum responsável por representar os perfis de acesso da aplicação, como USER e ADMIN, controlando diferentes níveis de permissão dentro do sistema.